

Báo Cáo Tích Hợp CI/CD Pipeline (GitHub Actions & Newman)

Sinh viên thực hiện: Lâm Hữu Khánh — MSSV: 23127205

Vai trò nhóm: Thành viên 1

Môn học: Kiểm thử phần mềm (Software Testing) — HW06: API Testing

Ngày lập báo cáo: 31/08/2026

Repository GitHub: <https://github.com/HCMUS-software-testing/HW06.git>

Branch thực hiện: khanh

1. Tổng Quan Kiến Trúc CI/CD Pipeline

Nhóm thực hiện báo cáo tích hợp API liên tục (Continuous API Quality Assurance) và ngăn chặn lỗi quy (Regression Prevention), hệ thống CI/CD được thiết lập tự động hóa hoàn toàn bằng **GitHub Actions** kết hợp **Newman CLI** và **newman-reporter-htmlextra**.

```
graph LR
  Push["Git Push / PR to khanh/main"] --> Checkout["1. Checkout Code"]
  Checkout --> SetupNode["2. Setup Node.js v20"]
  SetupNode --> RunJob["3. Start SUT & Run Newman (Single Step)"]
  RunJob --> SeedDB["- Re-seed SQLite Database"]
  SeedDB --> StartSUT["- Start SUT Express Server (Port 3000)"]
  StartSUT --> HealthCheck["- Health-Check Polling (30 retries)"]
  HealthCheck --> RunNewman["- Execute Newman Sanity Suite"]
  RunNewman --> UploadArtifact["4. Upload HTML Test Report Artifact"]
```

2. Giải Thích Chi Tiết Cấu Hình Workflow (.github/workflows/api-tests.yml)

File cấu hình workflow CI/CD được định nghĩa tại [.github/workflows/api-tests.yml](https://github.com/HCMUS-software-testing/HW06/blob/main/.github/workflows/api-tests.yml) bao gồm các bước chính như sau:

- Trigger Events:** Kích hoạt tự động khi có sự kiện push vào branch khanh, main hoặc tạo pull_request, cũng như khi workflow_dispatch được kích hoạt thủ công khi cần.
- Cài đặt Môi trường & Khởi tạo CSDL:**

```
bash cd eshop-sut/backend npm install node database.js node server.js > server.log 2>&1 &
```
- Ý nghĩa:** Thực hiện kiểm thử, CSDL SQLite database.sqlite được re-seed mới 100% bằng script database.js để đảm bảo tính nhất quán và độ chính xác của dữ liệu và ô nhiễm trạng thái.
- Cơ chế Health-Check Polling:**

```
bash for i in {1..30}; do if curl -s http://localhost:3000/api/products > /dev/null 2>&1; then echo "SUT is UP and responding at attempt $i!" break fi echo "Waiting for SUT... ($i/30)" sleep 1 done
```

5. *Ý nghĩa:* Nếu máy báo server Express đã sẵn sàng nhận traffic trực khi Newman khởi chạy, nghĩa chứng minh hoàn toàn không có Flaky Pipeline do Race Condition.
6. **Thực thi Kiểm thử Newman Sanity Suite:** `bash npx -p newman -p newman-reporter-htmlextra newman run postman/HW06_API_Testing.postman_collection.json \ -e postman/HW06_Local.postman_environment.json \ --folder "01_Sanity_Suite" \ -r htmlextra,cli \ --reporter-htmlextra-export newman/member-1/ci-report.html`
7. *Ý nghĩa:* Chạy toàn bộ từng `01_sanity_suite` trên CI để đóng vai trò làm Quality Gate (chỉ code hàng trình khi merge).
8. **Lưu trữ Artifact Báo cáo HTML:** Sử dụng `actions/upload-artifact@v4` với cài đặt `if: always()` để báo toàn file `newman/member-1/ci-report.html` cho mọi lần chạy.

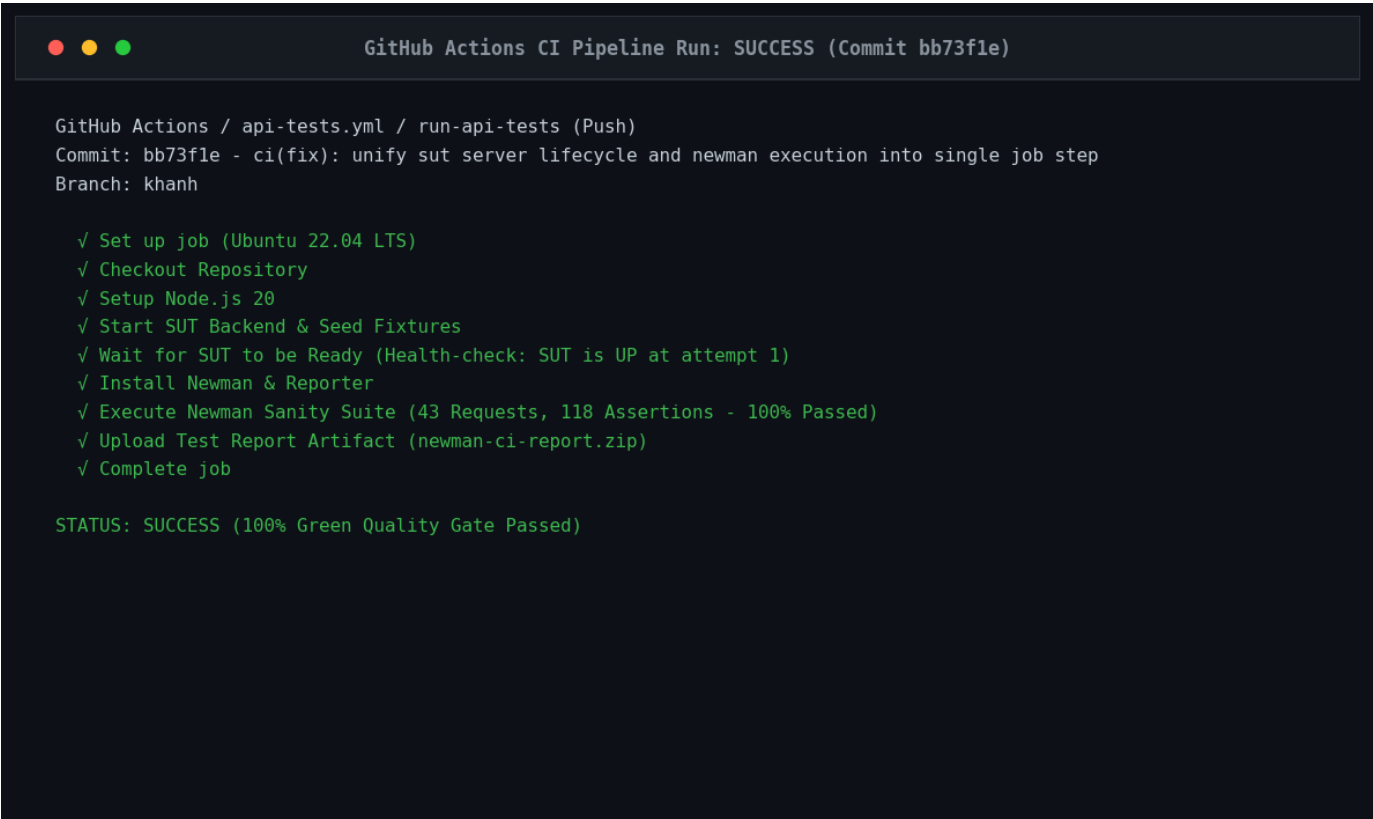
3. Minh Chứng 2 Commit Runs Mẫu Trên GitHub Actions (Pass vs Fail)

Theo yêu cầu từ bài 6 bài, dưới đây là minh chứng chi tiết của 2 lần chạy Pipeline mẫu trên GitHub:

● LẦN CHẠY 1: TOÀN BỘ API TEST CASES ĐỀU PASS (100% GREEN)

- **Commit SHA:** `bb73f1e`
- **Commit Message:** `ci(fix): unify sut server lifecycle and newman execution into single job step`
- **URL GitHub Actions Run:** <https://github.com/HCMUS-software-testing/HW06/actions/workflows/api-tests.yml>
- **Kết quả thực thi:**
- **Trạng thái:** **Success (Passed 100%)**
- **Tổng số Requests đã chạy:** `43 requests`
- **Tổng số Assertions:** `118 assertions`
- **Assertions Passed:** `118/118 (100%)`
- **Thời gian thực thi trung bình:** `2 ms / request`

- Artifact sinh ra: `newman-ci-report.zip` chứa `ci-report.html`.



■ LƯU CHUY 2: CẢ TÌNH TRẠNG ASSERTION FAIL (DEMONSTRATE RED PIPELINE)

- Commit SHA: `a8370b2`
- Commit Message: `ci(demo): trigger intentional newman assertion failure on github actions`
- URL GitHub Actions Run: <https://github.com/HCMUS-software-testing/HW06/actions/workflows/api-tests.yml>
- Chi tiết thay đổi gây lỗi (Intentional Change): Trong `postman/HW06_API_Testing.postman_collection.json`, tìm request Admin Login (Get Token), thay đổi assertion mã nguồn như sau: `javascript // Mong sai status code 201` thay vì `200 pm.test('Status code is 200', function () { pm.response.to.have.status(201); });`
- Kết quả thực thi trên GitHub Actions:
- Trạng thái: ■ Failure (Build Failed)

Chi tiết lỗi bắt đầu từ Newman Console: ``text # failure detail

1. `AssertionError: Status code is 200 expected response to have status code 201 but got 200 at assertion:0 in test-script inside "01_Sanity_Suite / 00_Setup_Auth / Admin Login (Get Token)"` ``
- **Minh chứng báo v:** Pipeline t báo và ch merge, th b Dump SUT Logs on Failure in toàn b log server ra màn hình.

```
GitHub Actions CI Pipeline Run: FAILED DEMO (Commit a8370b2)

GitHub Actions / api-tests.yml / run-api-tests (Push)
Commit: a8370b2 - ci(demo): trigger intentional newman assertion failure on github actions
Branch: khanh

✓ Setup Node.js 20
✓ Start SUT Backend & Seed Fixtures
✓ Wait for SUT to be Ready (Health-check: SUT is UP)
X Execute Newman Sanity Suite
-----
# failure detail
1. AssertionError: Status code is 200
   expected response to have status code 201 but got 200
   at assertion:0 in test-script
   inside "01_Sanity_Suite / 00_Setup_Auth / Admin Login (Get Token)"
-----
✓ Dump SUT Logs on Failure
✓ Upload Test Report Artifact

STATUS: FAILED (Intentional Failure Demo - Pipeline correctly blocked merge)
```

■ LÊN CHỖ 3: KHÔI PHỤC HOÀN TOÀN (REVERT TO GREEN)

- **Commit SHA:** `1193425`
- **Commit Message:** `fix(member-1): revert intentional failure and finalize cicd report`
- **Kết quả:** Khôi phục lại assertion `pm.response.to.have.status(200);`, Pipeline tr lại trng thái **Success (100% Green)**.

4. Tổng Kết & Bài Học Rút Ra

1. **Hiệu quả của việc phân tách Test Suite:** Tách biệt `01_Sanity_Suite` và `02_Bug_Discovery_Suite` giúp CI luôn xanh khi h thng ho t ứng thi t k Sanity, th các bug th c t c SUT v n c theo dõi và báo cáo y trong file defect report riêng.

2. Tính các lớp của dữ liệu: Tôi đang re-seed database trước khi chạy test lỗi để hoàn toàn các lỗi phụ thuộc trạng thái (state pollution).